

Airbag ECU Portfolio

LCP13 - Testing and Validation Plan / Test Report Framework

Measurement and Processing of Sensor Signals to Process an Airbag Operation

Field	Content
Project	Functional Safety - Airbag ECU, HS Harz FuSa
Document ID	LCP13-AirbagECU-TV-v1.0
Lifecycle focus	IEC 61508 life-cycle phases 7 and 13; verification evidence for phase 9 realization
Item	Airbag Electronic Control Unit prototype using Arduino Due / test board
Version / Date	1.00 / 2026-05-19
Prepared by	Project team - testing and validation
Review status	Draft for team review

1. Purpose and Scope

The purpose of this document is to define the testing and validation approach for the Airbag ECU prototype. The document explains how the implemented system will be checked against the defined safety goals, software requirements, hardware behavior, and expected system reactions. It connects each test activity with the related safety goal, safety parameter, expected behavior, and objective evidence. Therefore, the document does not only describe whether a test was performed, but also shows why the test is necessary from a functional safety point of view and which safety measure is being verified. This is important because safety-related systems must provide traceable evidence that the implemented functions behave correctly under normal operation, crash conditions, and fault conditions. [2], [5], [7]

The scope of this document covers the main validation activities required for the Airbag ECU prototype. This includes testing of the acceleration, gyroscope, and pressure sensor inputs; validation of the crash-decision logic; verification of passenger/baby-seat suppression; checking of digital output behavior for the driver airbag, passenger airbag, and warning lamp; and validation of CAN communication. In addition, the document covers fault handling, safe-state behavior, software module testing, integration testing, system-level validation, timing checks, output readback, code coverage, and requirement traceability. These activities are used to demonstrate that the prototype reacts safely in normal driving, valid crash scenarios, baby-seat conditions, sensor faults, output faults, and communication faults. [2], [3], [4], [8].

The main objective is to prove that the Airbag ECU does not activate the airbags unintentionally, deploys the required airbag output only when a valid crash and safing condition are detected, suppresses the passenger airbag when the baby-seat input is active, and enters a defined safe state when a fault is detected. The result of this document will be used as a test plan and test report framework for collecting evidence during prototype testing and validation.

2. Lifecycle Allocation

The lifecycle allocation follows the general E/E/PE safety lifecycle approach from IEC 61508-1 and the safety-related software lifecycle guidance from IEC 61508-3 [5], [6].

IEC 61508 Life-cycle phase	Meaning for this project	Output in this document
Phase 6 - Operation and maintenance planning	Define how the prototype should be checked before use and after faults.	Pre-test checks, proof-test ideas, maintenance/fault response criteria.
Phase 7 - Validation planning	Plan validation activities before running the final system tests.	Test strategy, pass/fail criteria, test environment, traceability matrix.
Phase 8 - Installation and commissioning planning	Define how the test board/Arduino wiring and CAN setup must be prepared.	Commissioning checklist and environment setup.
Phase 9 - E/E/PES realization	Implementation of sensors, logic, output control, readback, CAN, and fault handling.	Verification evidence from module and integration tests.
Phase 13 - Overall validation	Prove the final prototype reacts according to the safety requirements.	System validation scenarios and report framework.

3. Input Documents and Assumptions

Input	Used information
Project task sheet	Professor requires testing of sensor highest/lowest values, intended deployment values, output ON/OFF behavior, digital baby-seat input, CAN, code coverage, requirement tracing, V&V, and test reports.
HARA / ASIL document	Main hazards: unintended deployment, no deployment during crash, baby-seat suppression failure, delayed deployment, wrong airbag activation. Primary safety goals rated ASIL D or ASIL C.

Architecture document	Sensors, CPU, safing engine, watchdog, CAN/LIN, deployment drivers, backup power, safe-state handling.
Arduino software document	Inputs A0/A1/A2/D2, outputs D8/D9/D13, thresholds ACC >= 700, GYRO >= 600, PRESSURE >= 650, CAN frame 0x120, safe-state concept.

Standards basis: The HARA/ASIL link, system-level validation, software verification, requirement traceability, test planning, CAN communication, RAM test concept, and embedded C coding guidance are aligned with recognized published references [1]-[10].

4. Safety Goals and Safety Parameters

Safety goals and safety parameters are structured according to the concept-phase and safety-requirement approach used in ISO 26262-3 and ISO 26262-8 [1], [4].

ID	Safety goal	ASIL target	Safe state	Main safety parameters
SG1	Prevent unintended driver/passenger airbag deployment.	ASIL D	Deployment inhibited; warning lamp ON if fault exists.	Fail-safe behavior, diagnostic coverage, fault detection, output readback, independence.
SG2	Deploy required airbag(s) during a valid crash within the allowed reaction time.	ASIL D	If deployment cannot be guaranteed, fault shall be indicated and stored.	Availability, deterministic timing, FTTI/PST, redundancy, backup energy.
SG3	Suppress passenger airbag when baby seat/passenger suppression condition is active.	ASIL C	Passenger airbag OFF; driver airbag allowed if valid crash.	Safe input, input plausibility, priority logic, fail-safe suppression.
SG4	Activate the correct airbag output according to impact type and valid sensor combination.	ASIL D	Wrong output inhibited; fault indicated if selection is inconsistent.	Sensor plausibility, redundancy, cross-check, deterministic logic.
SG5	Detect and report sensor, output, CAN, memory, timing, or watchdog faults.	ASIL D support	Airbags OFF unless valid crash and safing path are confirmed; warning lamp ON.	DC, SFF support, fault containment, traceability, testability.

5. Step-by-Step Testing and Validation Strategy

The validation workflow follows system-level safety validation, software verification, and general software testing concepts from ISO 26262-4, ISO 26262-6, IEC 61508-1, and ISO/IEC/IEEE 29119-1 [2], [3], [5], [7].

Step	Action	Safety parameter satisfied
Step 1 - Requirement baseline	Freeze SRS/SWR IDs, thresholds, CAN frame, input/output pins, safe-state definition, and safety goals before test execution.	Traceability, QM, systematic error avoidance
Step 2 - Test environment commissioning	Check wiring, power, Arduino Due upload, sensor simulator/test board, CAN setup, serial monitor, output indicators, and fault lamp.	Installation correctness, repeatability
Step 3 - Sensor boundary testing	Set all sensor inputs to minimum, maximum, just-below-threshold, at-threshold, and invalid values.	Fault detection, diagnostic coverage, safe-state behavior
Step 4 - Functional crash testing	Apply intended crash values where airbag should unfold and verify correct outputs.	Safety function correctness, deterministic behavior
Step 5 - Baby-seat suppression testing	Repeat crash tests with baby-seat input active and inactive.	Priority logic, fail-safe passenger suppression

Step 6 - Fault injection	Inject sensor, output readback, CAN, timing, memory, and watchdog faults.	Fault tolerance, DC, fail-safe behavior
Step 7 - Software module testing	Test sensor manager, crash logic, deployment logic, CAN handler, fault handler, and safe-state function separately.	Branch/path coverage, modular verification
Step 8 - Integration testing	Test sensor-to-logic-to-output-to-CAN chain with realistic sequences.	Interface correctness, data flow, system integration
Step 9 - System validation	Run final use cases against safety goals and record evidence.	Validation, safety case evidence
Step 10 - Review and close findings	Record pass/fail results, deviations, corrective actions, and retest evidence.	Quality management, reproducibility

6. Test Environment and Commissioning Checklist

The commissioning checks support reproducible test execution, configuration control, and objective test evidence as expected by functional safety supporting processes [4], [5].

Item	Expected setup / check	Evidence
Controller	Arduino Due or equivalent test board configured with the Airbag ECU prototype software.	Board photo, software version, Git commit ID.
Analog inputs	ACC on A0, GYRO on A1, PRESSURE on A2; values adjustable from low to high range.	Serial monitor screenshot and measured ADC values.
Digital input	Baby-seat/passenger suppression switch on D2 with defined ON/OFF state.	Switch state screenshot/log.
Outputs	Driver airbag indicator D8, passenger airbag indicator D9, warning lamp D13.	LED/output measurement or logic analyzer screenshot.
CAN	CAN ID 0x120, DLC 8 bytes; normal/crash/fault messages visible.	CAN log or serial-simulated frame printout.
Power	Stable supply; optional backup capacitor/power-loss simulation if available.	Voltage reading and test notes.
Safety check	No real pyrotechnic actuator connected. Outputs represented only by LEDs/test loads.	Test setup sign-off.

7. General Pass/Fail Criteria

The pass/fail criteria are written to support objective verification and validation evidence for safety-related system and software requirements [2], [3], [7].

Criteria ID	Pass condition	Fail condition
AC-01	No airbag output is activated during normal driving or invalid input conditions.	Any unintended output activation.
AC-02	Driver airbag output activates only for valid crash plus safing condition and no blocking system fault.	Driver output missing during valid crash, or active without valid condition.
AC-03	Passenger airbag output is suppressed whenever baby-seat input is active.	Passenger output ON while baby-seat input is active.
AC-04	Warning lamp activates during detected fault state.	Fault exists but warning lamp remains OFF.
AC-05	CAN frame contains correct status, sensor values, deployment status, fault code, and checksum.	Wrong CAN ID, data length, missing fault code, or checksum mismatch not detected.
AC-06	Output readback matches commanded output, or mismatch causes safe state.	Mismatch exists and system continues normal operation.

AC-07	All safety-relevant requirements have at least one test case and recorded evidence.	Requirement without test evidence.
AC-08	Statement and branch coverage targets are documented and achieved for safety logic.	Coverage missing or untested decision branch remains.

8. Detailed Test Scenarios

8.1 Sensor Boundary and Plausibility Tests

Boundary and plausibility tests support safety validation by checking valid ranges, threshold behaviour, and diagnostic reactions for safety-related input signals [2], [3], [7].

Test ID	Scenario	Input values	Expected output	Safety parameter
SV-01	Normal driving baseline	ACC=300, GYRO=200, PRESSURE=250, Baby=OFF	D8 OFF, D9 OFF, D13 OFF, CAN normal	No unintended deployment
SV-02	Minimum acceleration	ACC=0, GYRO=200, PRESSURE=250	No deployment	Sensor range validation
SV-03	Maximum acceleration raw value	ACC=1023	Fault if outside valid range; lamp ON; airbags OFF	Fault detection / safe state
SV-04	ACC just below threshold	ACC=699, GYRO=650, PRESSURE=250	No frontal deployment	Boundary testing
SV-05	ACC at threshold	ACC=700, GYRO=650, PRESSURE=250	Valid frontal crash; driver ON; passenger ON if Baby=OFF	Correct threshold behavior
SV-06	Pressure just below threshold	PRESSURE=649, ACC=300, GYRO=650	No side crash deployment	Boundary testing
SV-07	Pressure at threshold	PRESSURE=650, ACC=300, GYRO=650	Valid side crash; deployment allowed if no baby seat	Correct threshold behavior
SV-08	Gyro below safing threshold	ACC=800, GYRO=599, PRESSURE=250	No deployment if safing requires GYRO >= 600	Safing validation
SV-09	Gyro at safing threshold	ACC=800, GYRO=600, PRESSURE=250	Deployment allowed if no fault and no baby-seat suppression	Safing validation

8.2 Crash Function Validation Tests

Test ID	Scenario	Input values	Expected output	Safety goal
CR-01	Valid frontal crash	ACC=800, GYRO=650, PRESSURE=300, Baby=OFF	Driver ON, Passenger ON, Lamp OFF, CAN crash status	SG2
CR-02	Valid side crash	ACC=300, GYRO=650, PRESSURE=800, Baby=OFF	Driver ON, Passenger ON or side output ON depending architecture; CAN crash status	SG2/SG4
CR-03	One high sensor only	ACC=800, GYRO=300, PRESSURE=250	No deployment if safing condition not met	SG1
CR-04	Angular/rollover value only	ACC=300, GYRO=700, PRESSURE=250	No deployment without crash input	SG1
CR-05	Both crash sensors active	ACC=850, GYRO=700, PRESSURE=850, Baby=OFF	Deployment allowed; CAN crash status; no warning	SG2/SG4
CR-06	Crash followed by normal state	Crash values then normal values	Outputs reset according to prototype logic; final state	State behavior

			logged	
--	--	--	--------	--

8.3 Passenger/Baby-Seat Suppression Tests

Test ID	Scenario	Input values	Expected output	Safety parameter
BS-01	Baby seat during frontal crash	ACC=800, GYRO=650, PRESSURE=300, Baby=ON	Driver ON, Passenger OFF, CAN passenger suppressed	Fail-safe suppression
BS-02	Baby seat during side crash	ACC=300, GYRO=650, PRESSURE=800, Baby=ON	Driver/allowed output ON, Passenger OFF	Priority logic
BS-03	No baby seat during crash	ACC=800, GYRO=650, Baby=OFF	Driver ON, Passenger ON	Correct normal deployment
BS-04	Baby seat, no crash	ACC=300, GYRO=200, PRESSURE=250, Baby=ON	Both airbags OFF	No unintended deployment
BS-05	Baby-seat input unstable/floating	Toggle Baby input or disconnect input	Fault or conservative passenger suppression; lamp ON if implemented	Input fault detection

8.4 Output Verification and Readback Tests

Test ID	Scenario	Method	Expected output	Implementation status
OUT-01	Driver output command/readback	Command D8 HIGH during valid crash and read back actual output.	Readback equals command; otherwise, safe state.	Planned / extend code
OUT-02	Passenger output command/readback	Command D9 HIGH only if Baby=OFF and read back.	Readback equals command; otherwise, safe state.	Planned / extend code
OUT-03	Warning lamp output	Inject sensor fault and verify D13.	D13 HIGH during fault.	Partly implemented
OUT-04	Output stuck-at HIGH fault	Force/read output HIGH when command is LOW.	Fault detected; deployment inhibited; lamp ON.	Planned / test-board required
OUT-05	Output stuck-at LOW fault	Command output HIGH but readback LOW.	Fault detected; safe state or diagnostic fault.	Planned / test-board required

8.5 CAN Communication Tests

CAN communication tests use ISO 11898-1:2024 as the external reference for CAN data-link-layer communication and message behaviour [8].

Test ID	Scenario	Input/condition	Expected CAN behavior	Safety parameter
CAN-01	Normal status frame	Normal sensor values	CAN ID 0x120, DLC 8, status normal, checksum correct	Communication integrity
CAN-02	Crash status frame	Valid crash scenario	Frame shows crash/deployment status and sensor values	Vehicle-level notification
CAN-03	Baby-seat suppression status	Valid crash + Baby=ON	Frame shows passenger airbag suppressed	Traceable safe action
CAN-04	Fault status frame	Invalid sensor or output mismatch	Frame shows fault code and warning status	Fault reporting
CAN-05	Wrong checksum received	Inject wrong checksum in received diagnostic/reset	Command rejected; CAN error/fault logged	Data integrity

		command		
CAN-06	Missing CAN communication	Disconnect/suppress CAN receive if applicable	Fault or diagnostic warning according to requirement	Fault detection

8.6 Fault Injection and Safe-State Tests

Fault-injection tests verify diagnostic reaction and safe-state behaviour. Memory-related tests are supported by the AUTOSAR RAM Test concept for checking RAM cell health [2], [5], [9].

Test ID	Fault injected	Method	Expected safe reaction	Safety parameter
FI-01	Sensor too high	Set ACC/GYRO/PRESSURE > 950 or invalid range.	Airbags OFF, warning lamp ON, CAN fault sent.	DC / safe state
FI-02	Sensor stuck-at low	Hold sensor at 0 while other inputs change.	No unintended deployment; plausibility fault if implemented.	Fault detection
FI-03	Sensor stuck-at high	Hold sensor above threshold constantly.	No deployment unless full valid crash+safing logic; fault if implausible.	Fault tolerance
FI-04	Output readback mismatch	Force output readback different from command.	Safe state; warning lamp ON.	Output diagnosis
FI-05	Watchdog/program freeze	Stop kicking watchdog or block main loop.	Reset or safe state; fault logged after restart.	Temporal monitoring
FI-06	Memory/config corruption	Modify threshold/config checksum intentionally.	CRC/check failure; safe state.	ROM/config integrity
FI-07	CAN corruption	Wrong ID, wrong DLC, wrong checksum.	Frame rejected and fault reported if required.	Data integrity
FI-08	Power interruption	Simulate power loss or voltage drop.	Backup energy maintains decision/logging if available; otherwise safe fault.	Availability

8.7 Timing / FTTI Validation Tests

Timing validation supports deterministic reaction of the safety function and checks that detected faults are handled before an unsafe state can persist [2], [5].

Test ID	Scenario	Measurement	Expected result	Safety parameter
TIM-01	Loop cycle time	Measure time from sensor read to output decision.	Within project-defined cycle time; no excessive delay.	Deterministic timing
TIM-02	Valid crash reaction time	Measure time from threshold crossing to output activation.	Below chosen FTTI/PST target for prototype.	FTTI/PST
TIM-03	Fault reaction time	Measure time from injected fault to lamp/safe state.	Fault detected before unsafe state can occur.	Fault detection time
TIM-04	CAN update time	Measure time from event to CAN status frame.	Crash/fault frame sent within selected project limit.	Communication timeliness

9. Software Module Test Plan

Software module tests are aligned with ISO 26262-6, IEC 61508-3, ISO/IEC/IEEE 29119-1 and MISRA C guidance for safety-related embedded software [3], [6], [7], [10].

Module	Main test objective	Example tests	Coverage target
--------	---------------------	---------------	-----------------

Sensor Manager	Read and validate ACC, GYRO, PRESSURE, baby-seat input.	Min/max/range/stuck-at tests.	Statement + branch
Crash Detection Algorithm	Convert raw sensor values into frontal/side/safing conditions.	Below, at, above threshold; one sensor high only.	Branch + path
Deployment Decision	Allow deployment only with crash, safing, no fault, and baby-seat rule.	All boolean combinations of crash/safing/fault/baby.	Decision/branch
Output Control	Set outputs and perform readback verification.	Command ON/OFF; mismatch injection.	Statement + branch
Fault Handler	Enter safe state for sensor/CAN/output/watchdog/memory faults.	All fault codes cause correct safe reaction.	Branch
CAN Handler	Build and validate 8-byte CAN frame.	Normal/crash/fault/checksum tests.	Statement + branch
Watchdog/Timing	Detect loop hang or timing error.	Freeze loop; delayed cycle test.	Path

10. Integration and System Validation

Integration and system validation are planned to connect module-level evidence to complete item behaviour and final safety validation [2], [3], [5].

Level	What is combined	Main validation question	Evidence
Module test	One software module at a time.	Does each module meet its software requirement?	Module test report, coverage log.
SW integration	Sensor manager + crash logic + deployment logic + fault handler.	Does the software chain make the correct decision?	Serial logs, test script output.
HW/SW integration	Arduino/test board + sensors + outputs + CAN.	Do real inputs produce correct real outputs?	Photos, videos, oscilloscope/logs, CAN logs.
Architecture verification	Architecture vs SRS and safety concept.	Does architecture include redundancy, diagnostics, safe state, CAN, backup power concept?	Review checklist.
System validation	Complete airbag ECU prototype.	Does the ECU react according to safety goals and safety requirements?	Final validation report.

11. Requirement-to-Test Traceability Matrix

Requirement-to-test traceability is treated as part of the supporting process and verification evidence according to ISO 26262-8 [4].

Req. ID	Requirement	Safety parameter	ASIL	Test cases
SWR-01	Read acceleration sensor input.	Input validity / diagnostic coverage	D support	SV-01 to SV-05, FI-01 to FI-03
SWR-02	Read gyroscope/angular sensor input.	Safing / timing / plausibility	D support	SV-08, SV-09, CR-01 to CR-05
SWR-03	Read pressure sensor input.	Side-impact detection / plausibility	D support	SV-06, SV-07, CR-02
SWR-04	Read passenger/baby-seat digital input.	Safe input / suppression priority	C	BS-01 to BS-05
SWR-05	Activate driver airbag for valid crash.	Correct functionality / deterministic logic	D	CR-01, CR-02, CR-05

SWR-06	Activate passenger airbag only if no baby seat is detected.	Fail-safe suppression	C	BS-01 to BS-04
SWR-07	Activate warning lamp during system fault.	Fault indication / safe state	D support	FI-01, FI-04, OUT-03
SWR-08	Send crash/fault status through CAN.	Data integrity / communication	D support	CAN-01 to CAN-04
SWR-09	Read CAN messages for diagnostic/reset command.	Communication validation / checksum	D support	CAN-05, CAN-06
SWR-10	Read back output status after activation.	Output diagnosis	D	OUT-01 to OUT-05, FI-04
SWR-11	Enter safe state if fault is detected.	Fail-safe behavior	D	FI-01 to FI-08
SWR-12	Support test routines for module/integration testing.	V&V / traceability	D support	All module, integration, and system tests

12. Code Coverage and Test Completeness Plan

Coverage planning supports software unit and integration verification by exercising safety-relevant statements, decisions, branches, representative paths, and defensive coding expectations [3], [6], [7], [10].

Coverage type	How it will be achieved	Acceptance target
Statement coverage	Run all module functions at least once: sensor read, crash evaluation, deployment decision, output control, fault handler, CAN handler.	100% for safety-relevant prototype code where practical.
Branch coverage	Test TRUE/FALSE outcomes of every if/else decision: crash/no crash, safing/no safing, baby/no baby, fault/no fault.	All safety-relevant branches executed.
Path coverage	Execute normal path, frontal crash path, side crash path, baby-seat path, fault path, CAN error path.	Documented representative paths.
Boundary coverage	Test just below, at, and above each sensor threshold.	All threshold boundaries verified.
Fault coverage	Inject stuck-at, out-of-range, readback mismatch, CAN corruption, watchdog/time faults.	Every specified fault leads to safe reaction or documented limitation.
Traceability coverage	Every SWR and SG has at least one linked test case.	No orphan requirement.

13. Test Report Template

The test report template is based on the need for objective, repeatable, and reviewable test evidence for validation and verification activities [4], [7]. The test report template is provided as a framework for educational validation. At the current project stage, no physical Airbag ECU test board, CAN analyzer, sensor simulator, or output measurement equipment was available to the team. Therefore, the test cases in this document define the planned validation method, input values, expected outputs, safety parameters, and required evidence. Actual results, pass/fail status, and evidence files shall be completed when the prototype is executed in a physical or approved simulation environment.

Field	Entry
Test ID	
Date / Tester	
Requirement(s)	
Safety parameter verified	

ASIL/SIL relevance	
Initial conditions	
Input values / sequence	
Expected result	
Actual result	
Evidence file name	
Pass / Fail	
Deviation / corrective action	
Retest result	

14. Example Test Execution Log

Test ID	Expected	Actual	Result	Evidence
SV-01	No deployment, lamp OFF.	To be filled during execution.	Open	serial_log_SV01.txt
CR-01	Driver ON, Passenger ON, CAN crash status.	To be filled during execution.	Open	video_CR01.mp4 / can_CR01.log
BS-01	Driver ON, Passenger OFF.	To be filled during execution.	Open	video_BS01.mp4
FI-01	Airbags OFF, warning lamp ON, CAN fault.	To be filled during execution.	Open	serial_FI01.txt

15. Open Issues and Required Improvements

Issue ID	Current gap	Impact	Required action
OI-01	Simple demonstration code does not fully implement physical output readback.	Cannot close SWR-10 with real evidence.	Add readback pins or test-board measurement and safe-state logic.
OI-02	CAN receive and checksum validation are described but not fully implemented in simple code.	CAN-05/CAN-06 cannot pass until implemented.	Add CAN receive handler and checksum verification.
OI-03	Watchdog and memory tests are mentioned but not implemented in simple code.	Timing/memory safety evidence incomplete.	Add watchdog initialization, CRC/config check, and startup self-test.
OI-04	Safing condition is inconsistent in documents: written logic mentions GYRO ≥ 600 OR ACC ≥ 700 , but code uses only GYRO ≥ 600 .	Test expectation may conflict with implementation.	Decide final safing rule and update both SRS and code consistently.
OI-05	Timing/FTTI target is not numerically defined for the prototype.	TIM tests cannot have clear pass/fail limits.	Define project-level timing target, e.g., max cycle/reaction time for prototype.

16. Final Validation Statement Template

The final validation statement shall only be used when objective evidence confirms that the Airbag ECU prototype satisfies the defined safety requirements and validation criteria. At the current project stage, the validation activities are prepared as an educational test and validation framework. Physical test execution was not performed because no real Airbag ECU test board, CAN analyzer, sensor simulator, or measurement equipment was available to the team.

Validation statement: Based on the defined test plan, the Airbag ECU prototype is expected to prevent unintended deployment, deploy the required airbag outputs only under valid crash and safing conditions, suppress the passenger airbag when the baby-seat input is active, enter a safe state for detected faults, and report status via CAN. Final

validation can only be confirmed after the planned test cases are executed and objective evidence such as logs, screenshots, CAN traces, or measurement records is collected. Remaining limitations and open issues are listed in Section 15.

17. Review and Approval

Role	Name	Date	Signature / Comment
Project Lead			
Functional Safety Manager			
Software Developer			
Hardware Developer			
Test Engineer			
Quality/Documentation Responsible			

19. References

- [1] International Organization for Standardization, ISO 26262-3:2018, Road vehicles - Functional safety - Part 3: Concept phase. Geneva: ISO, 2018. Available: <https://www.iso.org/standard/68385.html>
- [2] International Organization for Standardization, ISO 26262-4:2018, Road vehicles - Functional safety - Part 4: Product development at the system level. Geneva: ISO, 2018. Available: <https://www.iso.org/standard/68386.html>
- [3] International Organization for Standardization, ISO 26262-6:2018, Road vehicles - Functional safety - Part 6: Product development at the software level. Geneva: ISO, 2018. Available: <https://www.iso.org/standard/68388.html>
- [4] International Organization for Standardization, ISO 26262-8:2018, Road vehicles - Functional safety - Part 8: Supporting processes. Geneva: ISO, 2018. Available: <https://www.iso.org/standard/68390.html>
- [5] International Electrotechnical Commission, IEC 61508-1:2010, Functional safety of electrical/electronic/programmable electronic safety-related systems - Part 1: General requirements. Geneva: IEC, 2010. Available: <https://webstore.iec.ch/en/publication/5515>
- [6] International Electrotechnical Commission, IEC 61508-3:2010, Functional safety of electrical/electronic/programmable electronic safety-related systems - Part 3: Software requirements. Geneva: IEC, 2010. Available: <https://webstore.iec.ch/en/publication/5517>
- [7] ISO/IEC/IEEE, ISO/IEC/IEEE 29119-1:2022, Software and systems engineering - Software testing - Part 1: General concepts. Geneva: ISO, 2022. Available: <https://www.iso.org/standard/81291.html>
- [8] International Organization for Standardization, ISO 11898-1:2024, Road vehicles - Controller area network (CAN) - Part 1: Data link layer and physical coding sublayer. Geneva: ISO, 2024. Available: <https://www.iso.org/standard/86384.html>
- [9] AUTOSAR, Specification of RAM Test, AUTOSAR Classic Platform R24-11. AUTOSAR, 2024. Available: https://www.autosar.org/fileadmin/standards/R24-11/CP/AUTOSAR_CP_SWS_RAMTest.pdf
- [10] MISRA Consortium, MISRA C:2025 - Guidelines for the use of the C language in critical systems. MISRA, 2025. Available: <https://misra.org.uk/product-category/pdf-copies/>